

Algoritmos genéticos en aprendizaje de máquina

Resumen ejecutivo | Tres ejemplos ejecutables, un mismo ciclo evolutivo

Objetivo y trabajo realizado

Se implementaron y verificaron tres aplicaciones de algoritmos genéticos (AG): selección de características, optimización de hiperparámetros y búsqueda de arquitectura neuronal. Se revisaron dos cuadernos existentes, se creó el ejemplo faltante de selección de características y se documentó la ejecución en un README. Cada cuaderno contiene código, explicación, gráficos y resultados exportables.

Tres aplicaciones

01. Selección de características. Sobre Wine (178 observaciones, 13 variables y 3 clases), un cromosoma binario elige las entradas de una regresión logística. La aptitud combina exactitud de validación cruzada y una penalización de 0.02 por la fracción de variables utilizadas. Se comparó con el mismo modelo usando las 13 variables.

02. Hiperparámetros. Sobre Wine, un AG busca cinco parámetros de Random Forest: número de árboles, profundidad, mínimo de muestras para dividir, variables por división y criterio. La aptitud es la exactitud media de validación cruzada; la referencia es un Random Forest con parámetros predeterminados.

03. Neuroevolución. Sobre 300 puntos sintéticos con etiqueta $x_1 \times x_2 > 0$, se evoluciona una red de una o dos capas ocultas y activación ReLU o tanh. El AG elige arquitectura; MLPClassifier entrena los pesos con L-BFGS. La referencia es una capa de cuatro neuronas con tanh.

Método y evaluación

Los tres ejemplos explicitan representación, inicialización, aptitud, selección, cruzamiento, mutación y terminación. Se conservan dos élites. Las poblaciones son de 16, 10 y 8 individuos, con 10, 6 y 10 generaciones, respectivamente. Las semillas son 42 para los dos casos Wine y 10 para la red neuronal.

Wine se separa en 142 muestras de desarrollo y 36 de prueba; la selección usa CV estratificada de tres pliegues. La red utiliza 180 muestras de entrenamiento, 60 de validación y 60 de prueba. El test permanece fuera de la búsqueda; al final se reentrenan los modelos sobre todo desarrollo y se comparan con su referencia.

Código, cuadernos y documentación

GitHub: Miguel7293 / Algoritmos-gen-ticos-Actividad-02-

<https://github.com/Miguel7293/Algoritmos-gen-ticos-Actividad-02->

El README contiene enlaces directos para abrir los tres cuadernos en Google Colab, instrucciones y evidencias de ejecución.

Resultados verificados

Exactitud (%) de los modelos de referencia y de las soluciones seleccionadas.

Ejemplo	Referencia CV / val.	AG CV / val.	Referencia prueba	AG prueba
Características	97.19 %	100.00 %	97.22 %	97.22 %
Hiperparámetros	95.11 %	95.80 %	100.00 %	100.00 %
Neuroevolución	98.33 %	100.00 %	96.67 %	100.00 %

CV / val. es la métrica usada para seleccionar candidatos; prueba es una evaluación independiente. En características se muestra exactitud, no el fitness penalizado. Las tareas son distintas: estos valores no permiten ordenar los tres métodos entre sí.

Hallazgos principales

Características: se conservaron 7 de 13 variables (alcohol, ash, alkalinity_of_ash, flavanoids, color_intensity, hue, proline). La aptitud penalizada fue 0.9892. El cambio de exactitud en prueba frente a usar todas las variables fue de +0.00 puntos porcentuales.

Hiperparámetros: el ganador utilizó 80 árboles, profundidad 12, mínimo de división 3, max_features=log2 y criterio entropy. El cambio en prueba fue de +0.00 puntos porcentuales.

Neuroevolución: se eligieron capas ocultas [8, 2] con activación relu; se evaluaron 14 de las 18 arquitecturas posibles. El cambio en prueba fue de +3.33 puntos porcentuales. Se registraron 0 advertencias de convergencia durante la búsqueda y 0 en los ajustes finales.

Revisión y alcance de las conclusiones

Los tres cuadernos se ejecutaron completos en kernels nuevos de Jupyter, sin errores y con gráficos guardados. Se verificaron particiones independientes y conservación del mejor fitness por elitismo. Se corrigieron la ausencia de test en los ejemplos originales, el registro incompleto de la última generación de neuroevolución y la falta de una referencia de comparación.

Los ejemplos muestran cómo un AG busca soluciones, pero no garantizan mejor exactitud en prueba ni el óptimo global. Los datos son pequeños y se emplea una semilla por caso. La validación reutilizada durante la búsqueda puede sobreestimar el rendimiento. Para conclusiones generales harían falta varias particiones y semillas, además de comparar con búsqueda aleatoria o exhaustiva.

Trazabilidad y fuentes

Entorno verificado: Python 3.11.7, NumPy 2.4.6, scikit-learn 1.9.1 y Matplotlib 3.11.2. Las métricas e historiales están en resultados/*.json; los HTML del repositorio son material complementario con ejemplos propios.

Fuentes: consigna Actividad 02; scikit-learn: [validación cruzada](#); Wine; MLPClassifier.